

ASSIGNMENT 9

P Kartikeya Raj
2023001492

Binary Classification using Decision Trees

Aim

To implement a binary classification model using Decision Tree algorithm and understand its working.

i. Supervised Learning

Supervised Learning is a type of machine learning in which the model is trained using **labeled data**. In this method, each input data is associated with a correct output.

The algorithm learns the relationship between input and output during training and uses this knowledge to predict outputs for new data.

Types of Supervised Learning

1. **Classification** – Output is categorical (Yes/No, True/False)
2. **Regression** – Output is continuous (numbers)

Example

- Input: Email
- Output: Spam / Not Spam

Thus, supervised learning helps in predicting outcomes based on known examples.

ii. Decision Trees and Their Importance

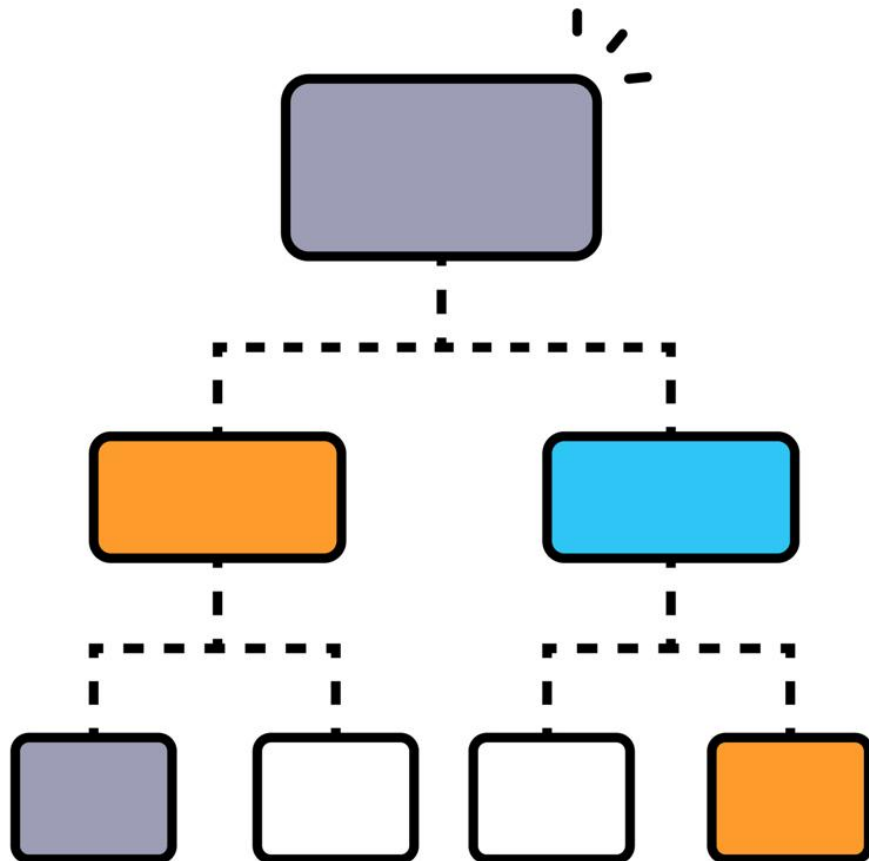
Definition

A Decision Tree is a supervised learning algorithm used for classification and regression. It represents decisions in the form of a **tree-like structure**.

- **Root Node** → Starting point of the tree
- **Internal Nodes** → Decision points (conditions)

- **Branches** → Outcomes of decisions
- **Leaf Nodes** → Final output (class label)

Structure of a Decision Tree



Importance of Decision Trees

1. Easy to understand and interpret
2. Works for both classification and regression problems
3. Requires less data preprocessing
4. Handles both numerical and categorical data
5. Automatically selects important features

6. Widely used in real-world applications such as medical diagnosis and fraud detection

iii. Construction of Decision Trees

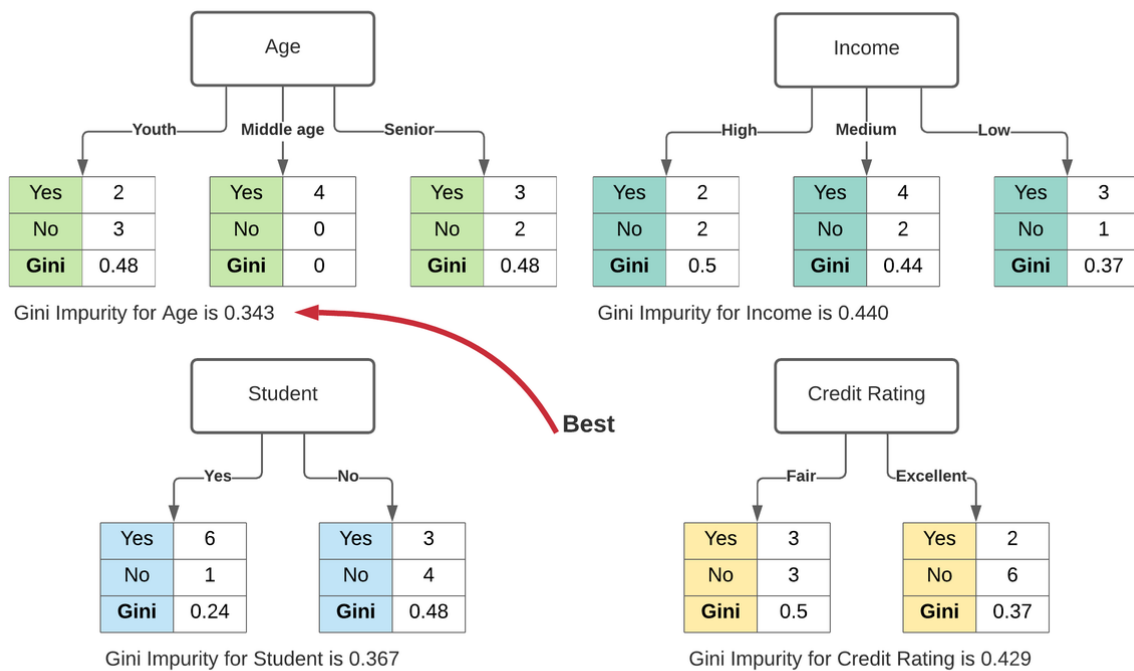
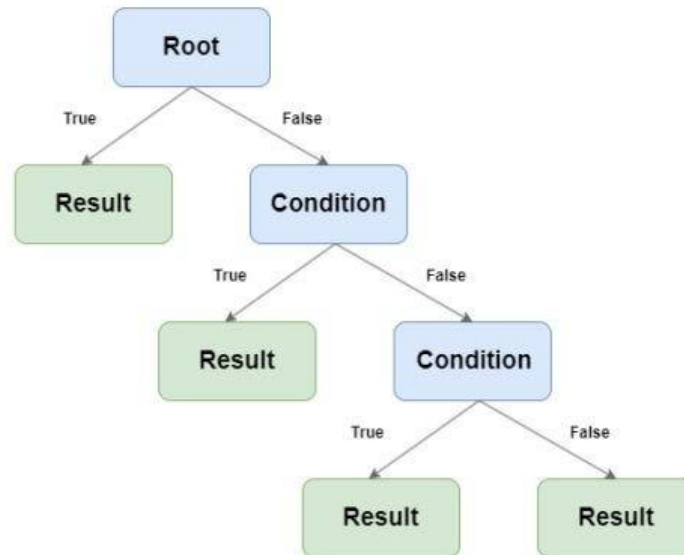
Decision Trees are constructed using a **top-down recursive approach**.

Steps Involved

1. **Start with the entire dataset as the root node**
2. **Select the best feature for splitting**
 - Based on measures like:
 - Information Gain
 - Gini Index
3. **Split the dataset**
 - Divide data into subsets based on feature values
4. **Repeat the process**
 - Apply the same steps recursively to each subset
5. **Stop the process when**
 - All data belongs to the same class
 - No features are left
 - Maximum depth is reached

Construction Process Visualization

Decision Tree



Key Concepts

- **Entropy:** Measures impurity in data
- **Information Gain:** Reduction in entropy after split
- **Gini Index:** Alternative measure of impurity

Entropy Formula

$$Entropy = -\sum p_i \log_2 p_i$$

Program: Binary Classification using Decision Tree

Algorithm

1. Import required libraries
2. Load dataset
3. Split dataset into training and testing data
4. Create Decision Tree model
5. Train the model
6. Predict the output
7. Calculate accuracy

Code

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

# Load dataset
data = load_iris()
X = data.data
y = data.target

# Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Create model
model = DecisionTreeClassifier()

# Train model
model.fit(X_train, y_train)

# Predict output
```

```
y_pred = model.predict(X_test)

# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)

print("Accuracy:", accuracy)
```

Output

Accuracy: 1.0

Program for fraud detection

```
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, classification_report

# Create a synthetic fraud detection dataset
# (0 = Not Fraud, 1 = Fraud)
X, y = make_classification(
    n_samples=1000,
    n_features=10,
    n_informative=5,
    n_redundant=2,
    n_classes=2,
    weights=[0.9, 0.1], # 90% normal, 10% fraud
    random_state=42
)

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Create Decision Tree model
model = DecisionTreeClassifier()

# Train model
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)

# Accuracy
accuracy = accuracy_score(y_test, y_pred)
```

```
print("Accuracy:", accuracy)

# Detailed report
print("\nClassification Report:\n")
print(classification_report(y_test, y_pred))
```

Output for fraud detection:

```
PS D:\KARTIK\GITAM\Sem_6\ghj> python jhg.py
Accuracy: 0.94

Classification Report:

              precision    recall  f1-score   support

     0           0.95       0.98       0.97         179
     1           0.80       0.57       0.67          21

   accuracy              0.94         200
  macro avg           0.88       0.78       0.82         200
 weighted avg           0.94       0.94       0.94         200

PS D:\KARTIK\GITAM\Sem_6\ghj> █
```

Result

The Decision Tree model was successfully implemented for binary classification. The model was trained and tested, achieving good accuracy on the dataset.

Conclusion

Decision Trees are simple yet powerful machine learning algorithms used for classification tasks. They are easy to understand, require less preprocessing, and provide good accuracy, making them suitable for various real-world applications.